

ELI — Memory

ELI v2.3.9

August 2026

Contents

ELI Memory Subsystem	1
Files	1
The Memory class (memory.py)	1
Schema (≈ 25 tables)	2
recall_memory — the hybrid retriever (memory.py:1722)	2
Vector store (vector_store.py)	3
Knowledge graph (knowledge_graph.py)	3
Truth layer (memory_truth.py)	3
Weight decay & consolidation	3
Honest assessment	3
Update — 2026-06-09	4
Update — 2.3.7 (recent-history window widened)	4

ELI Memory Subsystem

eli/memory/ — 7.0k LOC, 13 files. The persistent substrate: relational + full-text + vector + graph, all local SQLite/FAISS. Companion to project_overview.md.

Files

File	LOC	Role
memory.py	4.7k	the Memory god-class + DBPaths + module facade
knowledge_graph.py	643	entity/relation graph (KG)
habits_memory_db.py	470	
vector_store.py	430	FAISS vector index + embedder
__init__.py	0	
system_index.py	278	indexed apps/executables/files
memory_truth.py	190	
memory_adapter.py	131	compat adapter
memory_service.py,	small	helpers/compat
sqlite_memory.py, stores.py,		
populate_memories.py		

The Memory class (memory.py)

A single metaclass-backed class (~ 50 public methods) that owns essentially every persistent concern:

- **Semantic memory:** store_memory, add_memory, recall_memory, search_memory(ies), get_recent_semantic_memory, adjust_weight, apply_weight_decay.
- **Conversation:** add_conversation_turn, store_conversation, get_conversation_history, get_recent_conversations, get_recent_turns_since, search_conversations, get_turns_for_day, save_session_summary, get_session_summaries.
- **Habits:** log_habit_event, get_habit_events, add_habit_rule, get_habit_rules, record_habit_run.
- **Self-improvement / learning:** log_learning_event, log_failure, log_correction, add_observation, log_improvement, add_capability_proposal, propose_capability, get_pending_proposals, get_recent_failures/improvements, add_capability_proposal, propose_capability, get_pending_proposals, get_recent_failures/improvements.
- **Episodic/semantic/reflective aliases:** store_episodic, store_semantic, recall_semantic, store_reflective.
- **Stats / routing:** get_stats, get_dashboard_counts, get_db_routing_info.

vector_store is a lazy property; the KG is integrated via knowledge_graph.get_knowledge_graph().

Schema (≈25 tables)

memories (+ legacy memory), conversation_turns (+ conversations), session_summaries, kg_entities, kg_relations, habit_rules, habit_events, habits, failures, corrections, improvements, observations, capability_proposals, learning_replay, user_patterns, desktop_apps, executables, recent_files, user_dirs, error_tracking, recall_log, events, semantic, emotion_events. FTS5 virtual tables back conversation and KG search.

emotion_events (v2.1.31) — the emotional timeline

Owned by cognition/emotion_timeline.py, not the Memory class: it opens user.sqlite3 directly (same pattern as habits_memory_db.py) and creates its table idempotently, so it adds nothing to the 4.7k-line god-class.

column	meaning
ts, user_id, session_id	when / whose / which session
detected	what the USER seemed to feel
expressed	the register ELI answered in
valence	negative / positive / neutral — what makes “has been negative for a while” answerable without caring which specific emotion each read was
confidence, source, arousal	how strong the read was and where it came from (voice / text / fused / override)
user_text	the utterance that produced the read
eli_prior_action	the action ELI ran on the PRECEDING turn — this is the column that lets ELI ask whether the mood turned because of something <i>it</i> did

Indexed on ts and (user_id, ts). Reads come back ORDER BY ts DESC, id DESC — the id tiebreak matters because several reads can land inside one second and run-length counting would otherwise mis-order. See blueprints/perception.md for the assessment gates and the proactive surfacing path.

recall_memory — the hybrid retriever (memory.py:1722)

The shared retrieval foundation (see orchestration_and_agents.md for the two strategies on top):

1. **FAISS first** (Stage 5 vector primary). FTS5/LIKE runs only as a *supplement* when the vector index is empty/cold or returns < limit//2 hits. keyword_only=True skips FAISS entirely (the orchestrator runs its own semantic_search, so running FAISS here would double-search with a mislabeled source).

2. **Noise filtering** (important): excludes assistant_insight/episodic/ reflection kinds, orchestrator source, and reflection/assistant_insight/ session_summary tags, and rows longer than 1500 chars — so ELI's own old responses/reflections never resurface as “recalled user memories”. This was the fix for the “Immutable Techniques” contamination class of bug.
3. **Importance-weighted ordering** via COALESCE(importance, 0.5).

Heavy inline column-detection (`_memory_table_columns`) guards against schema drift across versions — defensive, but a sign the schema has churned.

Vector store (`vector_store.py`)

FAISS IndexFlat, embeddings via a local nomic embedder (`llama_cpp`). Notable: - `_embed_lock` (RLock) serializes embedding — the embedder is **not thread-safe** and concurrent calls segfault (this is why the orchestrator retrieval is sequential). - Metadata canonicalized to **meta.json** (migrated from legacy `meta.pkl`). - Singleton via `get_vector_store()`; shutdown-aware (skips embedding during teardown); `reset_vector_store()` for rebuilds.

Knowledge graph (`knowledge_graph.py`)

`kg_entities(name,type,aliases,description,confidence) + kg_relations(subject_id, predicate, object_id, weight, source)` — a subject-predicate-object graph. FTS5 over entities (with insert/update/delete triggers) for fuzzy search_entities. `upsert_entity`, `context_for_prompt` (lightweight SQLite-only prompt context — no embedding). Stop-word list prevents common words becoming entities.

Truth layer (`memory_truth.py`)

`inspect_sqlite / inspect_vector_store` — read-only authority/inspection used by status surfaces; reads `vectors/meta.json` preferentially, falls back to legacy pickle. Backs `truth_report` and the memory-status surfaces.

Weight decay & consolidation

`apply_weight_decay(decay_factor=0.98, min_weight=0.05, older_than_days=7)` — a multiplicative SQL update (`weight = MAX(min, weight*factor)` for old, low-importance rows). **There is no real episodic→semantic→KG consolidation**: decay is the only forgetting mechanism, and promotion across stores is manual (`store_episodic/store_semantic` are aliases, not a pipeline). This is a known gap ELI's own grounding admits.

Honest assessment

- **Strong**: genuinely hybrid (vector + FTS5 + KG) on one local SQLite foundation; the noise-filtering in `recall_memory` is the right instinct and fixed real contamination; embedder serialization correctly avoids the segfault.
 - **Weak**:
 1. `memory.py` is a **4.5k-line god-class** spanning ~8 unrelated concerns (semantic, conversation, habits, learning, failures, capabilities, system index). Wants to be split along those seams.
 2. **Schema sprawl / redundancy** — memories *and* memory, conversations *and* conversation_turns, plus a standalone semantic table. The inline column-detection everywhere is compensating for schema instability.
 3. **No consolidation pipeline** — only multiplicative decay; memories don't graduate into the KG automatically.
-

Update — 2026-06-09

- **Detected habits readable** (`Memory.get_detected_habits(min_count, limit)`): the proactive daemon fills the habits table via `detect_habits()`, but `HABIT_STATUS`, the persona overlay, and the bus `HabitAgent` all previously read only the (usually empty) `habit_rules` table → “no habits detected”. `get_detected_habits` reads the real habits table with a meta/introspection denylist (filters `SELF_REPORT/MEMORY_STATUS/EXPLAIN_*` noise from the user testing ELI), so genuine behaviour (media, app launches, screenshots) surfaces.
- **FAISS persistence bug fixed:** `_eli_persist_loaded_vector_store` was writing vector metadata with `pickle.dump` to `meta.json` while `_load_meta` reads JSON — so a rebuild corrupted the index and the next load silently auto-rebuilt (accumulating phantom vectors). Both write sites now use the canonical `_dump_meta` (JSON); the index re-syncs cleanly to the memory count. (This also closes a pickle-RCE-on-load vector the codebase had deliberately retired.) KG populator enriched (broader, clean entity extraction from `user_patterns`).

Update — 2.3.7 (recent-history window widened)

`runtime/memory_evidence.py` capped every recent-history pull at `max(4, min(limit, 8))`. The cap **silently ignored a larger limit**: a caller asking for 40 recent turns still received 8, so continuity was being thrown away by a constant nobody could see or configure.

- `RECENT_HISTORY_CAP = 40` — the ceiling now follows the caller, bounded only by a sane upper limit so a huge limit cannot drag the whole conversation table into a single turn.
- `collect_memory_evidence(limit=32)` and `build_memory_evidence_text(limit=32)` — defaults raised from 12 and 8.

This is separate from `cog.mem_recent_turns` (default 24, max 80), the user-facing tunable in Settings ► Cognition that governs how many recent turns enter the prompt. The prompt assembler still does the real budgeting downstream; this change only stops the evidence layer discarding history before the budgeter ever sees it.